Modelo de datos por motor

TP Match Making - Gaming Platform. Referencia para armar queries: nombres de colecciones, tablas, claves, nodos y campos en cada base.

Resumen

Motor	Base / espacio	Qué guarda	Conexión local (host)
MongoDB 8	DB `gaming`	Fuente de verdad: jugadores, ratings, catálogos (regiones, modos, temporadas), admins, reportes, sanciones y partidas.	`mongodb://gaming_admin:***@localhost:27017/gaming?authSource=admin`
Redis 7.4	DB 0	Estado en vivo: colas de matchmaking, partidas en curso, scoreboard, leaderboards.	`redis://localhost:6379`
Cassandra 5	keyspace `gaming`	Log de eventos por partida e historial de partidas por jugador.	`localhost:9042`, DC `datacenter1`
Neo4j 5	DB `neo4j`	Grafo de co-participación (jugador-partida) para detección de fraude.	`bolt://localhost:7687`, web `:7474`

Identificador compartido: `players._id` (UUID) es el mismo `playerId` en todos los motores (claves Redis, particiones Cassandra, nodos Neo4j). Lo mismo para `matches._id` = `matchId`.

1. MongoDB - base `gaming`

Todas las colecciones tienen validador `$jsonSchema` estricto (definido en `infra/mongo/init.js`). Fechas son `Date` salvo que se indique ms (epoch en milisegundos, número).

Colección	_id	Contenido
`players`	UUID	Perfil, rating por modo, standing por temporada
`matches`	UUID (matchId)	Partidas: participantes, cierre, scoreboard final
`sanctions`	UUID	Sanciones + journal de reversión de resultados
`reports`	UUID	Reportes de jugadores (manuales o automáticos)
`administrators`	UUID	Administradores / moderadores
`regions`	código (LATAM)	Catálogo estático: región -> países
`game_modes`	código (ranked_5v5)	Catálogo estático: modos y parámetros de matchmaking
`seasons`	código (2026-S2)	Catálogo estático: temporadas
`schema_metadata`	ObjectId	Versiones de esquema aplicadas (version, description, appliedAt)

players

Campo	Tipo	Descripción / valores
`_id`	string	UUID del jugador (playerId compartido)
`username`	string	3-32 chars, en minúsculas. Único
`email`	string	En minúsculas. Único
`passwordHash`	string	
`birthDate`	date	Opcional
`registeredAt / updatedAt`	date	
`country`	string	ISO alpha-2, ej. AR
`region`	string	regions._id derivado de country
`level`	int	>= 1
`accountStatus`	enum	active | inactive | closed (NO indica sanción)
`avatarUrl`	string	Opcional
`ratings.<modeId>`	object	{ mmr, gamesPlayed, updatedAt } - rating actual por modo. Ej: ratings.ranked_5v5.mmr
`seasonRatings.<seasonId>.<modeId>`	object	{ mmr, gamesPlayed, region, revision, updatedAt } - standing de leaderboard. gamesPlayed = 0 => no rankea
`appliedRatingMatches`	string[]	matchIds cuyo ajuste de rating ya se aplicó (idempotencia)
`revertedRatingMatches`	string[]	matchIds anulados cuyo ajuste ya se revirtió

Índices: username (único), email (único), {region, registeredAt}, country.

matches

Campo	Tipo	Descripción / valores
`_id`	string	matchId (UUID)
`mode`	string	game_modes._id
`region`	string	regions._id
`status`	enum	in_progress | closing | closed | voided
`ratingRange`	number	Rango de MMR aceptado al formarla
`createdAt`	date	
`players[]`	array	{ playerId, mmr, enqueuedAt (ms) }
`closure`	object	{ id, eventId, reason (normal|forfeit|administrative), requestedAt, tsMs (fin, ms), completedAt, results[]: { playerId, result: win|loss|draw } }
`scoringStatus`	enum	not_started | pending | calculating | applied | failed
`scoringUpdatedAt / scoringAttempts / scoringLastError`	date / int / string	
`finalScoreboard[]`	array	{ playerId, result, kills, deaths, objectives, score, mmrBefore, mmrAfter, ratingDelta }
`leaderboard`	object	{ status: published | not_ranked, updatedAt }
`voided`	object	{ sanctionId, at } - si la anuló una sanción

Índices: {status, createdAt}, {players.playerId, createdAt -1}.

sanctions

Campo	Tipo	Descripción / valores
_id	string	UUID
playerId / adminId	string	Jugador sancionado / admin que sanciona
reportId	string | null	Reporte que la originó
type	enum	matchmaking_restriction | temporary_suspension | permanent_ban
reason / description	string	
startsAt	date	
endsAt	date | null	null = permanente
status	enum	active | expired | revoked
createdAt	date	
matchIds	string[]	Partidas cuyos resultados anula
reversal	object	{ status: pending|reverting|completed|failed, attempts, completedSteps[]: queue|hold|leaderboard, heldBoards[]: {seasonId, mode, playerIds[]}, revertedMatchIds[], lastError, updatedAt, completedAt }

Vigente si status = 'active', startsAt <= now y (endsAt null o > now). Índices: {playerId, status, endsAt}, {playerId, createdAt -1}, reportId.

reports

Campo	Tipo	Descripción / valores
_id	string	UUID
reportedPlayerId	string	Jugador reportado
reporterPlayerId	string | null	null = reporte automático
matchId	string | null	
assignedAdminId	string | null	
reason	enum	cheating | match_fixing | toxicity | griefing | afk | other
description	string	
createdAt	date	
status	enum	open | assigned | resolved | dismissed

Índices: {reportedPlayerId, createdAt -1}, {status, createdAt}, matchId, createdAt.

administrators

Campo	Tipo	Descripción / valores
_id	string	UUID
username / email	string	Únicos
permissionLevel	enum	moderator | senior_moderator | admin
createdAt	date	
active	bool	

Catálogos estáticos

Campo	Tipo	Descripción / valores
regions._id / name	string	NA, LATAM, BR, EU, ASIA, OCE
regions.countries	string[]	Códigos ISO; cada país pertenece a una sola región (índice único)
regions.active	bool	
game_modes._id	string	ranked_1v1, ranked_5v5, casual_5v5, battle_royale
game_modes.name / description	string	
game_modes.minPlayers / maxPlayers	int	
game_modes.teams	int	0 = free for all
game_modes.ranked / active	bool	
game_modes.initialRating	number	1000
game_modes.matchmaking	object	{ initialRatingRange, ratingRangeStep, expandEverySeconds, maxRatingRange }

Campo	Tipo	Descripción / valores
seasons._id / name	string	2026-S1 (closed), 2026-S2 (active)
seasons.startsAt / endsAt	date	
seasons.status	enum	scheduled | active | closed (máximo una active)

Queries de ejemplo (mongosh)

```
use gaming
db.players.findOne({ username: "jugador1" })
db.players.find({ region: "LATAM" }, { username: 1, "ratings.ranked_5v5.mmr": 1 })
db.players.find().sort({ "seasonRatings.2026-S2.ranked_5v5.mmr": -1 }).limit(10)
db.matches.find({ "players.playerId": "<uuid>" }).sort({ createdAt: -1 })
db.matches.find({ status: "closed", mode: "ranked_5v5" }, { finalScoreboard: 1 })
db.sanctions.findOne({ playerId: "<uuid>", status: "active", startsAt: { $lte: new Date() },
                       $or: [{ endsAt: null }, { endsAt: { $gt: new Date() } }] })
db.reports.aggregate([{ $group: { _id: "$reportedPlayerId", n: { $sum: 1 } } },
                      { $sort: { n: -1 } }, { $limit: 10 }])
```

2. Redis - estado en vivo

No tiene esquema: son claves con convención de nombres. `{playerId}`, `{matchId}` son UUID; `{mode}` = game_modes._id; `{region}` = regions._id; `{seasonId}` = seasons._id (ej. 2026-S2).

Matchmaking (colas)

Clave	Tipo	Contenido
`mm:ticket:{playerId}`	HASH	ticketId, playerId, mode, region, mmr, enqueuedAt (ms). Uno por jugador.
`mm:q:{mode}:{region}:rating`	ZSET	member = playerId, score = mmr
`mm:q:{mode}:{region}:wait`	ZSET	member = playerId, score = enqueuedAt (ms)
`player:{playerId}:active_match`	STRING	matchId de la partida en la que está el jugador
`mm:formation:{matchId}`	HASH	status (reserved | committed | rolled_back), tickets (JSON), createdAt, abandoned:{playerId}
`mm:formations`	ZSET	Formaciones aún 'reserved', score = createdAt

Partidas en curso

Clave	Tipo	Contenido
`match:{matchId}`	HASH	matchId, mode, region, status=in_progress, startedAt, players (csv)
`match:{matchId}:scoreboard`	HASH	events, lastEventTs y por jugador: `{playerId}:kills|deaths|objectives|score|connected|connectionTs`
`match:{matchId}:applied`	SET	eventIds ya aplicados al scoreboard
`match:{matchId}:closure`	HASH	closureId, status=released, releasedAt. TTL 24 h

Leaderboards

Clave	Tipo	Contenido
`lb:{seasonId}:{mode}:global`	ZSET	member = playerId, score = MMR. Posición = ZREVRANK + 1
`lb:{seasonId}:{mode}:region:{region}`	ZSET	Igual, filtrado por región
`lb:{seasonId}:{mode}:members`	HASH	playerId -> "{revision}|{region}" (región vacía = tombstone)
`lb:{seasonId}:{mode}:rebuild_lock`	STRING	Token UUID del rebuild en curso. TTL 60 s
`lb:{seasonId}:{mode}:holds`	SET	"{sanctionId}|{playerId}" retenidos por una reversión
`lb:{seasonId}:{mode}:held`	HASH	playerId -> cantidad de reversiones que lo retienen

Comandos de ejemplo (redis-cli)

```
KEYS mm:q:*                                  # (solo dev) ver colas existentes
ZRANGE mm:q:ranked_5v5:LATAM:rating 0 -1 WITHSCORES
HGETALL mm:ticket:<playerId>
GET player:<playerId>:active_match
HGETALL match:<matchId>
HGETALL match:<matchId>:scoreboard
ZREVRANGE lb:2026-S2:ranked_5v5:global 0 9 WITHSCORES    # top 10
ZREVRANK  lb:2026-S2:ranked_5v5:global <playerId>         # posición - 1
ZREVRANGE lb:2026-S2:ranked_5v5:region:LATAM 0 9 WITHSCORES
```

3. Cassandra - keyspace `gaming`

Tablas diseñadas por query: siempre filtrar por la **partition key** y, opcionalmente, por las clustering keys en orden. Definidas en `infra/cassandra/init.cql`.

events_by_match

Log append-only de eventos de cada partida. PRIMARY KEY ((match_id), ts, event_id) - orden ts ASC, event_id ASC.

Columna	Tipo	Rol / descripción
`match_id`	uuid	**Partition key**
`ts`	timestamp	Clustering (ASC) - cuándo ocurrió
`event_id`	uuid	Clustering (ASC) - clave de idempotencia
`event_type`	text	kill | objective | disconnect | reconnect | match_finished
`player_id`	uuid	Autor (killer, quien tomó objetivo, quien se (des)conectó). null en match_finished
`target_player_id`	uuid	Solo en kill: la víctima
`detail`	text	JSON libre (en match_finished: datos del cierre)
`points`	int	Puntaje del evento

match_history_by_player

Historial de partidas por jugador, escrito al cerrar la partida. PRIMARY KEY ((player_id), ended_at, match_id) - orden ended_at DESC.

Columna	Tipo	Rol / descripción
`player_id`	uuid	**Partition key**
`ended_at`	timestamp	Clustering (DESC) - fin de la partida
`match_id`	uuid	Clustering (ASC)
`mode / region`	text	Modo y región de la partida
`result`	text	win | loss | draw
`kills / deaths / objectives / score`	int	Estadísticas del jugador en esa partida
`mmr_before / mmr_after / rating_delta`	int	Rating antes, después y diferencia

Queries de ejemplo (cqlsh)

```
USE gaming;
SELECT * FROM events_by_match WHERE match_id = <uuid>;
SELECT * FROM events_by_match WHERE match_id = <uuid>
  AND ts >= '2026-11-01 00:00:00+0000' AND ts <= '2026-11-02 00:00:00+0000';
SELECT * FROM match_history_by_player WHERE player_id = <uuid> LIMIT 20;
SELECT * FROM match_history_by_player WHERE player_id = <uuid>
  AND ended_at >= '2026-10-01 00:00:00+0000';
-- Ojo: filtrar por columnas que no son de la clave requiere ALLOW FILTERING (evitar).
```

4. Neo4j - grafo de co-participación

Modelo: `(:Player)-[:PLAYED]->(:Match)`. Se carga desde partidas cerradas (problema 7); una sanción con reversión borra el nodo Match (DETACH DELETE).

Elemento	Propiedad	Tipo	Descripción
`:Player`	`id`	string	playerId (UUID). Constraint único player_id_unique
`:Match`	`id`	string	matchId (UUID). Constraint único match_id_unique
`:Match`	`mode`	string	game_modes._id
`:Match`	`region`	string	regions._id
`:Match`	`endedAtMs`	integer	Fin en epoch ms. Índice match_ended_at
`[:PLAYED]`	`result`	string	win | loss | draw
`[:PLAYED]`	`ratingDelta`	integer	Cambio de rating en esa partida

Queries de ejemplo (Cypher)

```
MATCH (p:Player {id: $playerId})-[r:PLAYED]->(m:Match)
RETURN m.id, m.mode, r.result, r.ratingDelta ORDER BY m.endedAtMs DESC LIMIT 20;

// Pares de jugadores enfrentados repetidamente (posible match fixing)
MATCH (a:Player)-[ra:PLAYED]->(m:Match)<-[rb:PLAYED]-(b:Player)
WHERE a.id < b.id AND m.endedAtMs >= $sinceMs
  AND ((ra.result = 'win' AND rb.result = 'loss') OR (ra.result = 'loss' AND rb.result = 'win'))
WITH a, b, count(m) AS matches, sum(CASE WHEN ra.result = 'win' THEN 1 ELSE 0 END) AS winsA
WHERE matches >= 3
RETURN a.id, b.id, matches, winsA, matches - winsA AS winsB ORDER BY matches DESC;
```